Our goal is to isolate the Invoice class completely from TaxJar—including its specific exception classes. That's why we transform any Taxjar::Error::NotFound error we see into an error type we have defined.

The rest of the application no longer needs to have any knowledge of TaxJar. If TaxJar's API changes, this class is the only piece of code we'll have to update.

When we create a new Invoice, we now pass in the wrapper instead of the original TaxJar class:

15-using-test-doubles-effectively/24/sales_tax/lib/invoice.rb

```
def initialize(address, items, sales_tax: SalesTax.new)
  @address = address
  @items = items
  @sales_tax = sales_tax
end
```

Here's what the new, simpler Invoice#tax_rate method looks like:

15-using-test-doubles-effectively/24/sales_tax/lib/invoice.rb

```
def tax_rate
  @sales_tax.rate_for(@address.zip)
end
```

Our unit spec for Invoice is also much easier to understand and maintain. We no longer need to construct a rickety structure of test doubles. A single instance double will do:

15-using-test-doubles-effectively/24/sales_tax/spec/unit/invoice_spec.rb

```
require 'invoice'

RSpec.describe Invoice do
  let(:address) { Address.new(zip: '90210') }
  let(:items) { [Item.new(cost: 30), Item.new(cost: 70)] }

  it 'calculates the total' do
➤   sales_tax = instance_double(SalesTax, rate_for: 0.095)

    invoice = Invoice.new(address, items, sales_tax: sales_tax)

    expect(invoice.calculate_total).to eq(109.50)
  end
end
```

By wrapping the TaxJar API, we've improved our code and specs in many ways:

- The wrapper API is simpler to call, because it omits details we don't need.
- We can switch to a different third-party sales tax service by changing just one class, leaving the rest of our project intact.